Ejada Egypt Summer Internship Program 2026

Week 2 Technical Documentation

Prepared by: Yara Adel Khedr
github link: https://github.com/Yara-Khedr/ejada-intern-terraform

Contents

1. Introduction

1.1 Purpose

This document covers the design, deployment, and validation of the Week 2 network: a two-tier OCI environment with a public Load Balancer and a private application server backed by File Storage, built with Terraform.

1.2 Environment

Attribute	Value
Tenancy	ociejada
Region	me-jeddah-1
Compartment	intern-17-yara-khedr-cmp
Terraform / OCI CLI	1.15.8 / 3.90.1
Terraform Code (Week 2)	https://github.com/Yara-Khedr/ejada-intern-terraform/tree/main/week2

Table 1.1: Environment summary

Resources follow the naming pattern `Yara<Resource>02`, marking them as Week 2 builds.

2. Architecture Diagram

Full network architecture – VCN, subnets, gateways, Load Balancer, compute, and File Storage – including the Bastion.

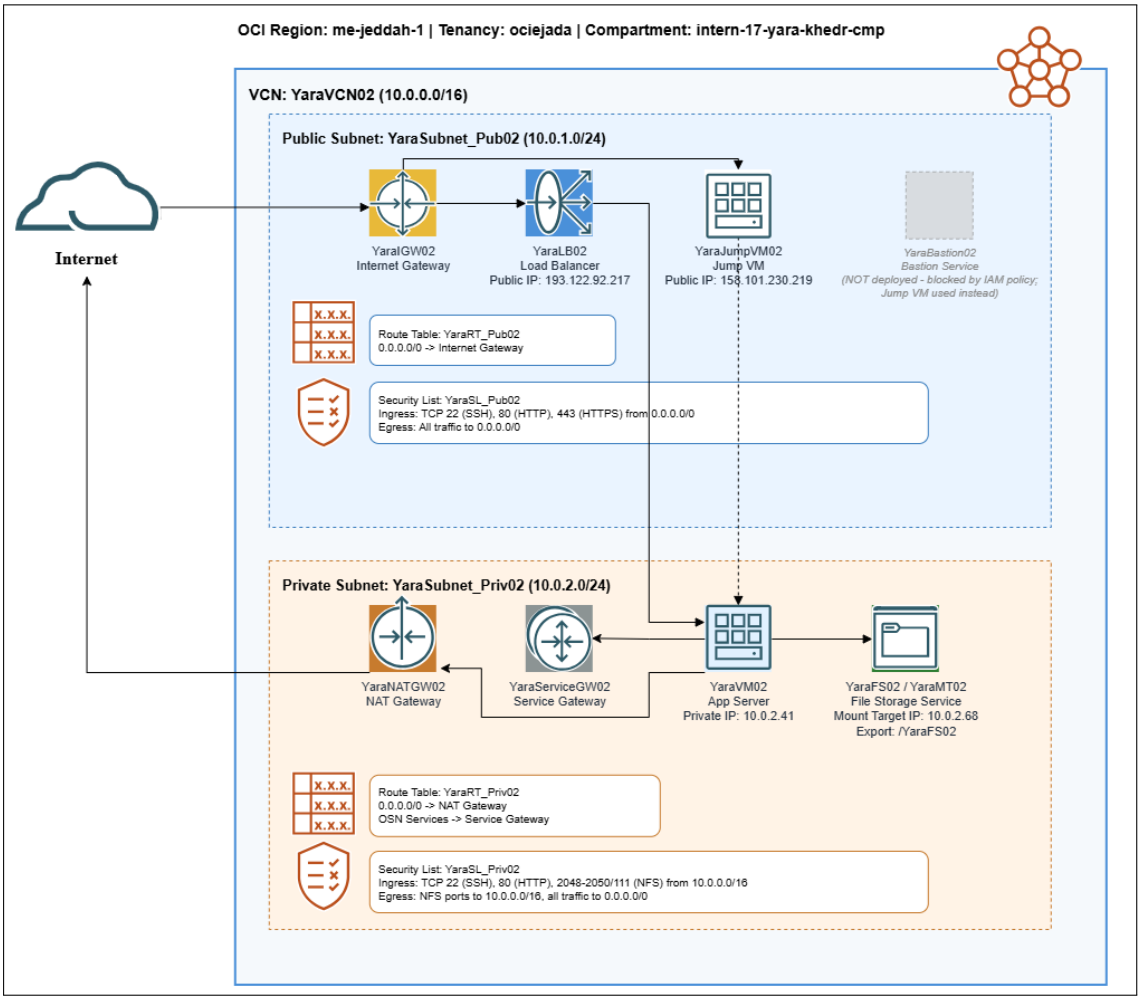

Figure 2.1: Week 2 OCI network architecture

3. Network Architecture

3.1 VCN and CIDR

`YaraVCN02` uses `10.0.0.0/16`, split into two `/24` subnets.

Network	CIDR	Purpose
YaraVCN02	10.0.0.0/16	VCN
YaraSubnet_Pub02	10.0.1.0/24	Public tier
YaraSubnet_Priv02	10.0.2.0/24	Private tier

Table 3.1: CIDR allocation

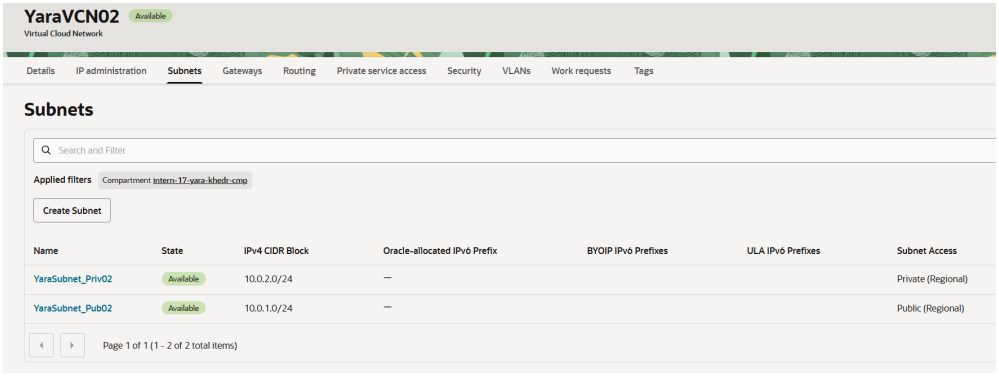

Figure 3.1: VCN YaraVCN02 with both subnets.

3.2 Public Subnet

Hosts **YaraLB02** (Load Balancer, `193.122.92.217`) and **YaraJumpVM02** (`158.101.230.219`). Routed via `YaraRT_Pub02` → Internet Gateway.

3.3 Private Subnet

Hosts **YaraVM02** (app server, private only) and **YaraMT02** (FSS mount target). No public IPs. Inbound traffic only via the Load Balancer or Jump VM; outbound via NAT Gateway.

3.4 Gateways

Gateway	Role
YaraIGW02 (Internet GW)	Public subnet in/out internet access
YaraNATGW02 (NAT GW)	Outbound-only internet access for private subnet
YaraServiceGW02 (Service GW)	Private access to OCI services over Oracle backbone

Table 3.2: Gateway roles

3.5 Route Tables

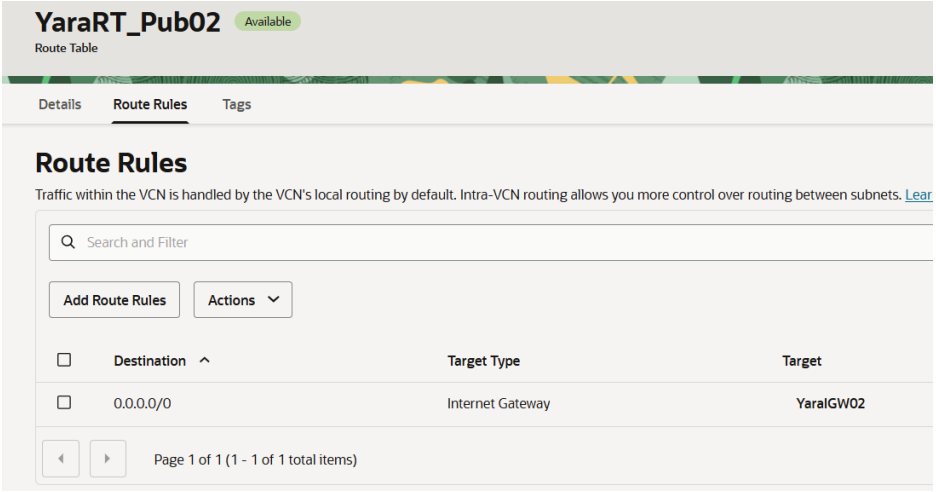

Figure 3.2: YaraRT_Pub02 – routes to Internet Gateway.

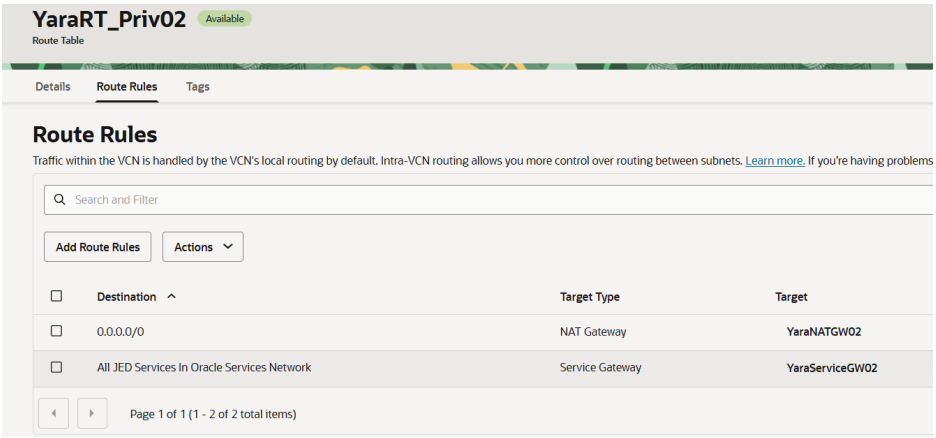

Figure 3.3: YaraRT_Priv02 – routes to NAT Gateway and Service Gateway.

4. Security Design

4.1 Security Lists vs. NSGs

Security Lists were used instead of NSGs: each subnet has one clear role, so subnet-level rules are sufficient. NSGs suit cases where instances in the *same* subnet need different rules, which doesn't apply here.

4.2 Public Security List

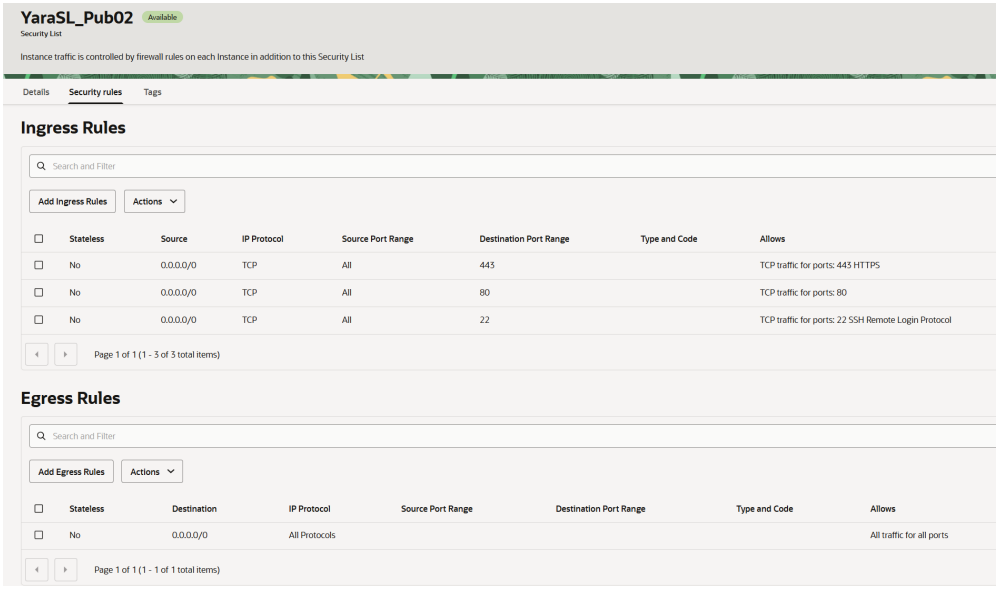

Figure 4.1: YaraSL_Pub02 – ingress 443/80/22 from 0.0.0.0/0; egress all.

4.3 Private Security List

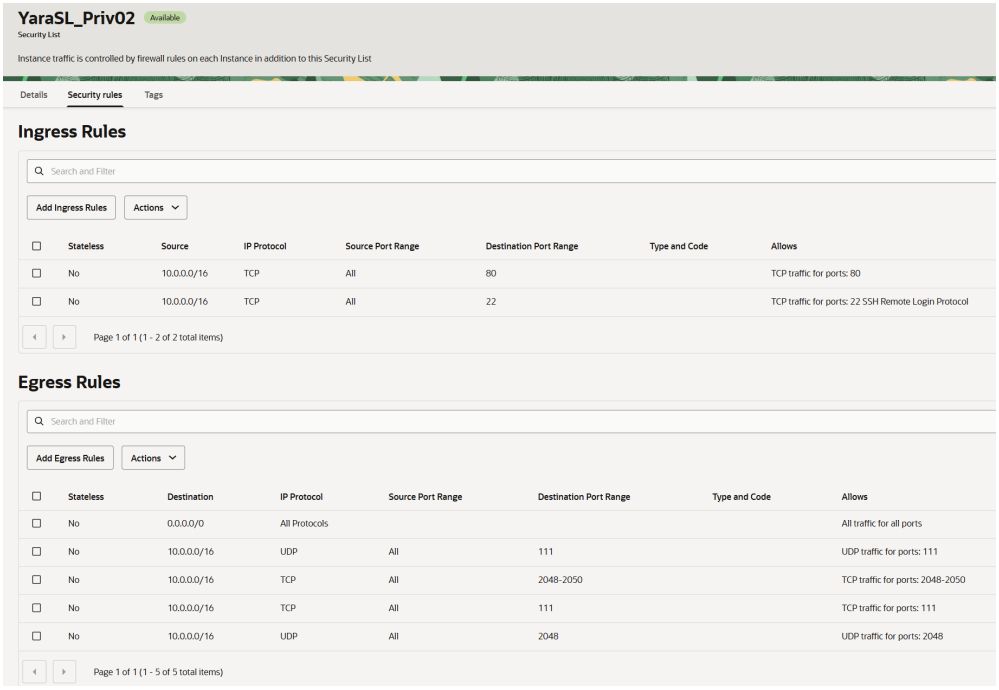

Figure 4.2: YaraSL_Priv02 – ingress restricted to 10.0.0.0/16; egress includes NFS ports.

Design Note

Private-subnet ingress is limited to `10.0.0.0/16` so `YaraVM02` only accepts traffic originating inside the VCN – never directly from the internet.

5. Compute, Application Tier and Access Model

5.1 YaraVM02

Oracle Linux 9, `VM.Standard.A1.Flex` (1 OCPU, 6 GB), private subnet, no public IP.

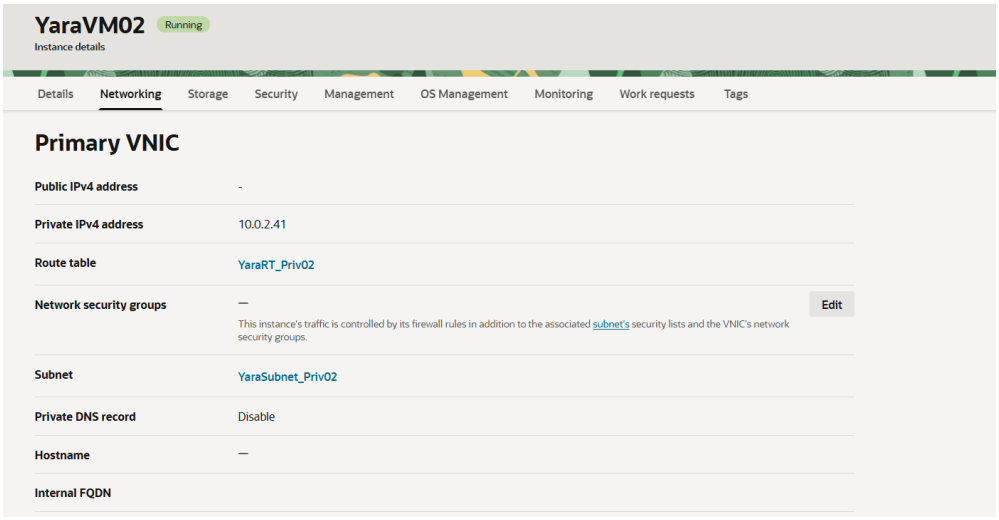

Figure 5.1: YaraVM02 – private IP 10.0.2.41.

Cloud-init handles setup on boot: installs `nfs-utils`/`httpd`, mounts FSS at `/mnt/appdata` (persisted via `/etc/fstab`), writes `index.html`, opens port 80 in `firewalld`, and enables `httpd`.

5.2 Secure Access Model

Design intent: OCI Bastion Service (session-based, no permanent host).
Blocker: `YaraBastion02` failed – IAM policy in the compartment prevented Bastion provisioning.
Resolution: YaraJumpVM02 deployed in the public subnet as the SSH entry point, restricted by `YaraSL_Pub02`/`YaraSL_Priv02` to keep the same network-level guarantee – no direct path from the internet to `YaraVM02`.

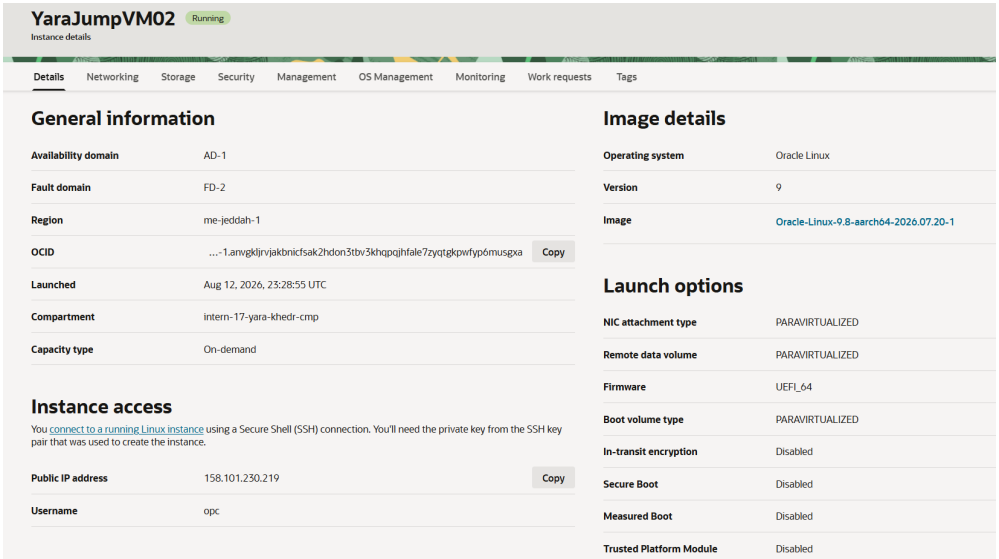

Figure 5.2: YaraJumpVM02 – public IP 158.101.230.219.

Method	**Status**
Cloud Shell	Available for quick checks
Jump VM	**Used** – Terraform-managed SSH relay
Bastion Service	Designed, blocked by IAM
SSH Tunneling	Used through the Jump VM

Table 5.1: Private-VM access options

6. File Storage Service

`YaraFS02` (file system) is exposed via mount target `YaraMT02` (`10.0.2.68`) through export `/YaraFS02`.

Figure 6.1: YaraMT02 mount target – private IP 10.0.2.68.

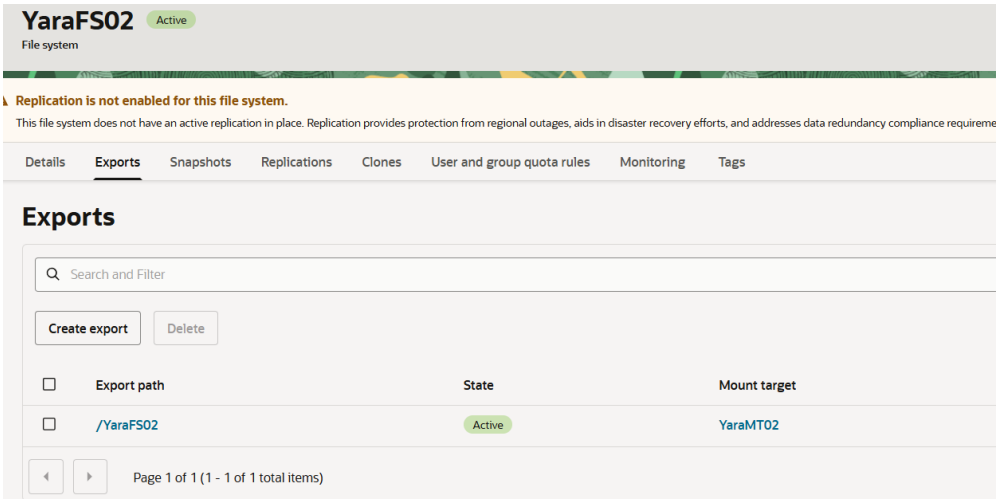

Figure 6.2: YaraFS02 export – path /YaraFS02.

Mounted automatically by cloud-init at `/mnt/appdata`; NFS ports (111, 2048–2050) are opened between the private subnet and mount target via `YaraSL_Priv02`.

7. Load Balancer

`YaraLB02`: Flexible shape, 10–100 Mbps, public IP `193.122.92.217`.
Listener `YaraListener02` (port 80) forwards to backend set `YaraBES02` (ROUND_ROBIN), backend = `YaraVM02`:80.

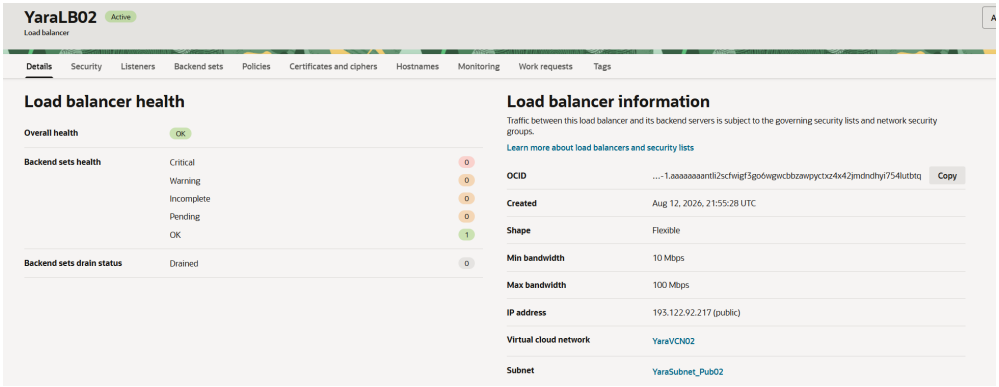

Figure 7.1: YaraLB02 – health OK, one backend healthy.

Browsing the public IP returns the page served by `YaraVM02`, confirming the full path works end-to-end.

8. Terraform Implementation

8.1 File Structure

Code lives in `week2/` (Week 1 untouched in `week1/`): https://github.com/Yara-Khedr/ejada-intern-terraform/tree/main/week2

File	Purpose
`providers.tf`	OCI provider config
`variables.tf` / `terraform.tfvars`	Inputs
`data.tf`	Data source lookups
`locals.tf`	Derived values
`main.tf`	Resources
`outputs.tf`	Outputs
`cloud-init.sh`	Boot-time app setup (templated)

Table 8.1: File structure – week2/

8.2 Variables, Locals, Data Sources

Values that vary by environment – compartment ID, subnet CIDRs, instance sizing, Load Balancer configuration, and the application port – are pulled from variables. Locals are used for values derived from data sources: the Service Gateway's service ID and CIDR, and the availability domain name. Display names and protocol-level constants (TCP/UDP numbers, NFS ports) remain as literals, since they define the naming convention and protocol behavior rather than environment-specific configuration.

Variable	Purpose
`region`, `compartment_id`	Target region / compartment
`vcn_cidr`, `allowed_ingress_cidr`	VCN sizing and public ingress scope
`public_subnet_cidr`, `private_subnet_cidr`	Subnet sizing per tier
`instance_shape/ocpus/memory_gbs`	App server compute sizing
`jump_vm_ocpus`, `jump_vm_memory_gbs`	Jump VM compute sizing
`ssh_public_key_path`	Instance SSH access
`fss_export_path`	NFS export path for File Storage
`lb_shape/min_bandwidth/max_bandwidth`	Load Balancer shape and bandwidth range
`app_port`	Application port (backend, health check, listener)
`bastion_allowed_cidr`	Reserved for Bastion (unused – IAM blocked)

Table 8.2: Key variables

Local	Purpose
`all_services_id`	Oracle Services Network OCID for Service GW
`all_Services_cidr`	Associated CIDR label for route rules
`availability_domain_name`	AD used for compute placement

Table 8.3: Key locals

8.3 Resource Summary

Type	Name	Purpose
VCN	YaraVCN02	10.0.0.0/16
Subnet	YaraSubnet_Pub02	Public, 10.0.1.0/24
Subnet	YaraSubnet_Priv02	Private, 10.0.2.0/24
Internet GW	YaraIGW02	Public internet access
NAT GW	YaraNATGW02	Outbound-only for private tier
Service GW	YaraServiceGW02	Private access to OCI services
Route Table	YaraRT_Pub02	Public routing
Route Table	YaraRT_Priv02	Private routing
Security List	YaraSL_Pub02	Public firewall rules
Security List	YaraSL_Priv02	Private firewall rules
Compute	YaraVM02	App server (private)
Compute	YaraJumpVM02	Jump host (public)
File System	YaraFS02	App data storage
Mount Target	YaraMT02	NFS access point
Export	YaraExport02	/YaraFS02
Load Balancer	YaraLB02	Public entry point
Backend Set	YaraBES02	ROUND_ROBIN
Listener	YaraListener02	HTTP :80
Bastion	YaraBastion02	Written, commented out (IAM blocked)

Table 8.4: Resource inventory

8.4 Outputs

Output	Value
`lb_public_ip`	193.122.92.217
`vm_private_ip`	10.0.2.41
`jump_vm_public_ip`	158.101.230.219
`mount_target_ip`	10.0.2.68
`fss_export_path`	/YaraFS02

Table 8.5: Terraform outputs

```
Apply complete! Resources: 2 added, 0 changed, 2 destroyed.

Outputs:

fss_export_path = "/YaraFS02"
jump_vm_public_ip = "158.101.230.219"
lb_public_ip = "193.122.92.217"
mount_target_ip = "10.0.2.68"
vm_private_ip = "10.0.2.41"
```

Figure 8.1: Final terraform apply – success, all outputs.

9. Issues Encountered

9.1 Bastion Blocked by IAM

Cause: IAM policy in the compartment prevented Bastion provisioning.
Fix: Jump VM deployed as the working alternative.

9.2 firewalld Blocking Port 80

Cause: Oracle Linux 9 ships with `firewalld` active, blocking inbound port 80 by default – independent of OCI Security List rules.

```
firewall-cmd --permanent --add-service=http
firewall-cmd --reload
```

```
[opc@YaraVM02 ~]$ sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
success
success
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: enp0s6
  sources:
  services: dhcpv6-client http ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
[opc@YaraVM02 ~]$
```

Figure 9.1: firewalld fix applied on YaraVM02.

Lesson

Network-layer rules (Security Lists) and host firewalls are independent – both must allow the traffic.

9.3 Cloud-init Mount Hang

Cause: Instance placed in the wrong subnet has no route/permission to the FSS mount target, so the NFS mount hangs at boot.
Fix: confirmed `YaraVM02` launches into `YaraSubnet_Priv02` only.

9.4 Diagnosing Without Bastion or Path Analyzer

Console history only showed early boot output, not cloud-init logs, and Network Path Analyzer was blocked by the same IAM restriction as the Bastion.
Fix: diagnosed manually through the Jump VM – reading `/var/log/cloud-init-output.log` directly via SSH, and running `tcpdump` to confirm traffic was reaching `YaraVM02`, which isolated the port-80 issue to the guest firewall.

10. Testing and Validation

10.1 Connectivity

SSH to `YaraVM02` verified via Jump VM; unreachable directly from the internet, as designed.

10.2 Load Balancer

Backend health OK: browser request to the public IP returns the app page.

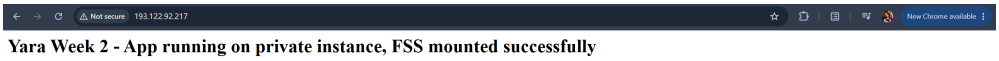

Figure 10.1: End-to-end verification via `http://193.122.92.217`.

10.3 FSS Mount

Export confirmed mounted and writable at `/mnt/appdata`; `/etc/fstab` entry persists the mount across reboots.

11. Conclusion

Week 2 delivered a two-tier OCI network, fully defined in Terraform: a public Load Balancer, a private application server, and File Storage, all verified working end-to-end.

Beyond the deployment itself, this week reinforced a few broader principles that carry into any cloud environment: security must be enforced at multiple layers, not just one; real environments come with constraints (IAM, permissions, service limits) that require practical workarounds rather than blocking progress; and when high-level tools aren't available, first-principles troubleshooting still gets the job done.